

PySpark Performance Tuning & Optimization Guide

Joins, skew, AQE, memory and cluster sizing in production Spark

A practical reference for diagnosing and fixing the performance problems that actually show up in production Spark jobs: the wrong join strategy, a handful of skewed keys holding up a stage, shuffle partitions left at the default, and executors sized by guesswork. Every section states the symptom, the cause, the fix, and how to confirm the fix worked.

Vivek Pandey — Data Engineer
Azure · Databricks · PySpark · Delta Lake
vp171097.github.io/DevWebsite

Contents

1. How Spark decides to run your query
2. Join strategies and broadcast tuning
3. Data skew: detection and salting
4. Adaptive Query Execution (AQE)
5. Partitioning: repartition, coalesce and shuffle sizing
6. Caching, persistence and memory layout
7. Executor and cluster sizing
8. File layout and the small-file problem
9. A diagnostic workflow
10. Configuration quick reference

1. How Spark decides to run your query

Every optimization below is an attempt to influence one of three things: how much data moves across the network (shuffle), how evenly that data is distributed across tasks (skew), and how much of it has to be read at all (pruning). Before tuning anything, it helps to know which of the three you are fighting.

Spark compiles a DataFrame into a logical plan, optimizes it with Catalyst, and produces a physical plan of stages separated by shuffle boundaries. Tasks within a stage run in parallel; a stage cannot finish until its slowest task finishes. That last sentence explains most Spark performance problems.

```
# The plan is the source of truth – read it before changing any config.
df.explain("formatted")      # physical plan with per-node detail
df.explain("cost")           # includes row-count / size estimates when stats exist

# What to look for:
#   Exchange hashpartitioning  -> a shuffle; expensive, try to remove or shrink
#   BroadcastHashJoin          -> good: small side was broadcast
#   SortMergeJoin              -> both sides shuffled and sorted
#   BroadcastNestedLoopJoin    -> usually a bug: non-equi join on a large table
#   PhotonScan / FileScan      -> check PushedFilters and PartitionFilters
```

Reading the physical plan is faster than guessing which config to change.

Rule of thumb If a config change does not alter the physical plan, it will not change performance. Verify the plan before and after.

2. Join strategies and broadcast tuning

Spark picks a join strategy from the size estimates it has. When those estimates are wrong — stale statistics, a filter it cannot see through, a UDF in the predicate — it picks badly, and you get a shuffle-heavy sort-merge join where a broadcast would have done.

Strategy	When Spark chooses it	Cost	Use when
Broadcast Hash Join	One side under autoBroadcastJoinThreshold (default 10 MB)	No shuffle of the large side	Dimension table joined to a fact table
Sort Merge Join	Both sides large, join keys sortable	Shuffle + sort both sides	Two large fact tables
Shuffle Hash Join	One side much smaller but above broadcast threshold	Shuffle both, hash the small side	Medium/large joins where sorting is wasteful
Broadcast Nested Loop	Non-equi join (>, BETWEEN, range)	$O(n*m)$ — very expensive	Avoid; rewrite as equi-join or range-bucket first

Raising the broadcast threshold

```
# Default is 10 MB. On a cluster with healthy executor memory, 64-128 MB is
# usually safe and converts many sort-merge joins into broadcast joins.
spark.conf.set("spark.sql.autoBroadcastJoinThreshold", 128 * 1024 * 1024)

# Force it when you know the size and Spark's estimate is wrong:
from pyspark.sql.functions import broadcast
fact.join(broadcast(dim_customer), "customer_id", "left")

# Disable broadcasting entirely (diagnosing OOM on the driver):
spark.conf.set("spark.sql.autoBroadcastJoinThreshold", -1)
```

Watch out The broadcast side is collected to the driver before being sent to executors. Broadcasting a table larger than the driver heap produces a driver OOM, not a slow job. Size the threshold against **spark.driver.memory**, not executor memory.

Statistics are what make the automatic choice work. On Delta tables, collect them so Catalyst has real numbers instead of file-size guesses:

```
ANALYZE TABLE portfolio.ecom_silver.orders COMPUTE STATISTICS FOR ALL COLUMNS;

-- Verify what the optimizer believes:
DESCRIBE EXTENDED portfolio.ecom_silver.orders;
```

3. Data skew: detection and salting

Skew is when one or a few join keys carry a disproportionate share of rows. After the shuffle, those keys land in one partition, one task processes them, and the whole stage waits. The symptom is unmistakable in the Spark UI: 199 tasks finish in seconds, one runs for 40 minutes, and it is often the one that spills to disk or fails with an OOM.

Confirming skew before fixing it

```
from pyspark.sql import functions as F

# 1. Key distribution — is one value dominant?
(df.groupBy("customer_id").count()
 .orderBy(F.desc("count"))
 .show(20, truncate=False))

# 2. Null keys are the classic hidden skew: every null hashes to one partition.
df.filter(F.col("customer_id").isNull()).count()

# 3. Actual partition sizes after the shuffle:
(df.withColumn("pid", F.spark_partition_id())
 .groupBy("pid").count()
 .orderBy(F.desc("count"))
 .show(10))
```

In the Spark UI, compare the max task duration against the median for the stage. A ratio above ~5x is skew.

Fix 1: let AQE handle it

On Spark 3.x this should be the first thing you try, because it requires no code change. AQE detects partitions much larger than the median at runtime and splits them across multiple tasks.

```
spark.conf.set("spark.sql.adaptive.enabled", True)
spark.conf.set("spark.sql.adaptive.skewJoin.enabled", True)
spark.conf.set("spark.sql.adaptive.skewJoin.skewedPartitionFactor", 5)
spark.conf.set("spark.sql.adaptive.skewJoin.skewedPartitionThresholdInBytes", "256MB")
```

Limitation AQE skew handling applies to sort-merge and shuffle-hash joins. It does not fix skew in a **groupBy** aggregation, and it cannot help when a single key is larger than the threshold on its own. That is when you salt.

Fix 2: salting the skewed key

Salting spreads one hot key across N partitions by appending a random suffix, then explodes the small side to match every suffix. The join becomes N smaller joins that run in parallel.

```
from pyspark.sql import functions as F

SALT_N = 32

# Large, skewed side: append a random salt to the join key.
fact_salted = fact.withColumn(
    "salt", (F.rand() * SALT_N).cast("int")
).withColumn("join_key", F.concat_ws("_", F.col("customer_id"), F.col("salt")))

# Small side: replicate every row once per salt value.
dim_salted = (dim
    .withColumn("salt", F.explode(F.array([F.lit(i) for i in range(SALT_N)])))
    .withColumn("join_key", F.concat_ws("_", F.col("customer_id"), F.col("salt"))))

result = (fact_salted.join(dim_salted, "join_key", "left")
    .drop("salt", "join_key"))
```

Cost: the small side is duplicated N times. Keep N as low as it can be while still flattening the skew.

Fix 3: salt only the hot keys

Replicating the whole dimension for the sake of three bad keys is wasteful. Isolate them instead:

```
hot = [k[0] for k in (fact.groupBy("customer_id").count()
    .filter("count > 1000000")
    .select("customer_id").collect())]

is_hot = F.col("customer_id").isin(hot)

# Salted path for hot keys, plain join for everything else, then union.
hot_result = salted_join(fact.filter(is_hot), dim, SALT_N)
cold_result = fact.filter(~is_hot).join(F.broadcast(dim), "customer_id", "left")
result = hot_result.unionByName(cold_result)
```

Null-key skew

```
# Nulls never match in an inner join, so filter them out before the shuffle.
matched = fact.filter(F.col("customer_id").isNotNull()).join(dim, "customer_id")

# For a left join, keep them but separate them from the shuffle:
nulls = fact.filter(F.col("customer_id").isNull())
result = matched.unionByName(nulls, allowMissingColumns=True)
```

4. Adaptive Query Execution (AQE)

AQE re-plans the query between stages using statistics measured from the data that actually flowed, rather than estimates made before execution. It is enabled by default from Spark 3.2 and does three useful things.

Feature	What it does	Config
Coalesce shuffle partitions	Merges small post-shuffle partitions so you stop paying for thousands of tiny tasks	<code>spark.sql.adaptive.coalescePartitions.enabled</code>
Skew join handling	Splits oversized partitions across several tasks at runtime	<code>spark.sql.adaptive.skewJoin.enabled</code>
Dynamic join switching	Demotes a planned sort-merge join to a broadcast join once the real size is known	<code>spark.sql.adaptive.autoBroadcastJoinThreshold</code>

```
spark.conf.set("spark.sql.adaptive.enabled", True)
spark.conf.set("spark.sql.adaptive.coalescePartitions.enabled", True)

# Target size per post-shuffle partition. 64-128 MB is a sound default.
spark.conf.set("spark.sql.adaptive.advisoryPartitionSizeInBytes", "128MB")

# Floor on partition count after coalescing – keep it >= total cores
# so the cluster does not sit idle.
spark.conf.set("spark.sql.adaptive.coalescePartitions.minPartitionNum", 200)
```

Reading AQE plans With AQE on, `df.explain()` shows the initial plan. The plan that actually ran appears in the Spark UI SQL tab, marked **AdaptiveSparkPlan isFinalPlan=true**. Judge results from the UI, not from `explain()`.

5. Partitioning: repartition, coalesce and shuffle sizing

`spark.sql.shuffle.partitions` defaults to 200 regardless of your data size or cluster. For a 10 GB shuffle that is 200 x 50 MB tasks, which is reasonable. For a 2 TB shuffle it is 200 x 10 GB tasks, which will spill and may never finish. With AQE enabled, set it high and let coalescing bring it down.

Operation	Shuffle?	Can increase partitions?	Typical use
<code>repartition(n)</code>	Yes — full shuffle	Yes	Rebalance before a wide operation or write
<code>repartition(col)</code>	Yes — hash partitioned	Yes	Co-locate rows by key before a join or window
<code>coalesce(n)</code>	No — merges locally	No	Reduce file count just before writing
<code>repartitionByRange(col)</code>	Yes — sampled ranges	Yes	Ordered output, range-friendly layout

```
# Sizing rule: aim for 128-200 MB of shuffled data per partition.
# partitions ~= shuffle_bytes / 134_217_728
spark.conf.set("spark.sql.shuffle.partitions", 1024) # AQE will coalesce down

# coalesce() avoids a shuffle but limits parallelism upstream: coalesce(1)
# after a heavy transform forces that transform onto a single task.
# Prefer this when you need one output file:
df.repartition(1).write.mode("overwrite").parquet(path) # shuffle, full parallelism upstream
# over:
df.coalesce(1).write.mode("overwrite").parquet(path) # no shuffle, serialized upstream
```

Partitioning the table vs partitioning the DataFrame `partitionBy()` on a writer creates directory-level partitions on disk. `repartition()` controls in-memory task distribution. They are unrelated; conflating them produces either thousands of tiny files or a single enormous one.

6. Caching, persistence and memory layout

Caching is only worth it when a DataFrame is consumed more than once and recomputing it is more expensive than storing it. Cached data competes with execution memory, so a cache that is never reused actively slows the job down.

Storage level	Stored as	Notes
<code>MEMORY_AND_DISK</code>	Deserialized in memory, spills to disk	Default for DataFrames; safest choice
<code>MEMORY_ONLY</code>	Deserialized in memory	Partitions that do not fit are recomputed, not spilled
<code>MEMORY_AND_DISK_SER</code>	Serialized (compact)	2-4x smaller, costs CPU to deserialize
<code>DISK_ONLY</code>	On local disk	For very large intermediates reused a few times

```
from pyspark import StorageLevel

enriched = orders.join(customers, "customer_id").filter("status = 'delivered'")
enriched.persist(StorageLevel.MEMORY_AND_DISK)
enriched.count()          # cache is lazy – force materialization

daily   = enriched.groupBy("order_date").sum("revenue")
by_city = enriched.groupBy("city").sum("revenue")

enriched.unpersist()      # release it as soon as the last consumer is done
```

Memory fractions

```
# Unified memory: execution (shuffle/sort/join) and storage (cache) share one pool.
spark.conf.set("spark.memory.fraction", 0.6)          # share of heap for both
spark.conf.set("spark.memory.storageFraction", 0.5)    # cache's protected share

# Symptom: "Container killed by YARN for exceeding memory limits"
# -> off-heap overhead too small for Python workers / Arrow buffers
spark.conf.set("spark.executor.memoryOverhead", "3g")
```

PySpark specifically Python UDFs run in a separate Python process; that memory lives in **memoryOverhead**, not the JVM heap. Heavy pandas UDFs need overhead raised well above the 10% default. Prefer built-in functions or pandas UDFs over row-at-a-time Python UDFs — the serialization cost dominates.

7. Executor and cluster sizing

The common failure is one extreme or the other: a handful of very large executors that lose long GC pauses, or hundreds of single-core executors that cannot share broadcast data.

```
# Balanced starting point: 4-5 cores per executor.
# - more than 5 -> HDFS/object-store throughput per executor degrades
# - fewer than 3 -> broadcast data is duplicated across too many JVMs

# Example: 10 worker nodes, 16 cores and 64 GB each
# 1 core + 1 GB per node reserved for the OS/daemons -> 15 cores, 63 GB usable
# 3 executors per node x 5 cores = 30 executors, 150 cores
# 63 GB / 3 = 21 GB per executor, minus ~10% overhead -> 18 GB heap
spark.conf.set("spark.executor.cores", 5)
spark.conf.set("spark.executor.memory", "18g")
spark.conf.set("spark.executor.memoryOverhead", "3g")

# Default parallelism should be 2-3x total cores so tasks stay short.
spark.conf.set("spark.default.parallelism", 150 * 2)
```

Dynamic allocation

```
spark.conf.set("spark.dynamicAllocation.enabled", True)
spark.conf.set("spark.dynamicAllocation.minExecutors", 2)
spark.conf.set("spark.dynamicAllocation.maxExecutors", 40)
spark.conf.set("spark.dynamicAllocation.executorIdleTimeout", "60s")
# Required so shuffle files survive executor removal:
spark.conf.set("spark.shuffle.service.enabled", True)
```

8. File layout and the small-file problem

A job that reads 200,000 files of 200 KB each spends most of its wall-clock time on file listing and task scheduling, not on computation. The target is files of roughly 128 MB to 1 GB.

```
-- Delta: compact small files and co-locate on high-cardinality filter columns
OPTIMIZE portfolio.ecom_silver.orders
  WHERE order_date >= current_date() - INTERVAL 7 DAYS
  ZORDER BY (customer_id, product_id);

-- Let Databricks size files automatically
ALTER TABLE portfolio.ecom_silver.orders
  SET TBLPROPERTIES (delta.autoOptimize.optimizeWrite = true,
                    delta.autoOptimize.autoCompact = true);
```

Partition columns Partition on a low-cardinality column that appears in most WHERE clauses — typically a date. Partitioning by a high-cardinality column such as `customer_id` creates one directory per customer and reintroduces the small-file problem at the worst scale. Use Z-ORDER or liquid clustering for high-cardinality access paths instead.

9. A diagnostic workflow

Work symptom-first. Changing configs without a hypothesis is how tuning sessions consume days.

Symptom	Likely cause	First thing to try
One task far slower than the rest	Key skew	AQE skew join; then salt the hot keys
Thousands of tiny tasks	shuffle.partitions too high for the data	Enable AQE coalescing
Few very slow tasks, heavy spill	Partitions too large	Raise shuffle.partitions
Driver OutOfMemory	Broadcast too large, or collect() on a big DataFrame	Lower broadcast threshold; avoid collect()
Executor killed by the resource manager	memoryOverhead too small	Raise executor.memoryOverhead
Stage re-runs repeatedly	Lost shuffle files after executor loss	Enable external shuffle service
Job slow but cluster idle	Not enough partitions to fill the cores	repartition to 2-3x total cores
Scan reads far more than expected	Predicate not pushed down	Check PushedFilters; remove UDFs from the predicate

1. Read the physical plan and find the widest exchange.
2. Open the Spark UI and compare max vs median task time for the slowest stage.
3. Check spill-to-disk and GC time columns before touching any memory config.
4. Change exactly one setting, re-run, and compare the same two numbers.
5. Record what worked in the job's README — the next person will hit the same wall.

10. Configuration quick reference

Setting	Default	Sensible production value
spark.sql.shuffle.partitions	200	2-3x total cores, with AQE coalescing on
spark.sql.adaptive.enabled	true (3.2+)	true
spark.sql.adaptive.advisoryPartitionSizeInBytes	64MB	128MB
spark.sql.autoBroadcastJoinThreshold	10MB	64-128MB (bounded by driver memory)
spark.sql.adaptive.skewJoin.enabled	true	true
spark.executor.cores	1 / all	4-5
spark.executor.memoryOverhead	10% of heap	15-20% for PySpark workloads
spark.memory.fraction	0.6	0.6
spark.sql.files.maxPartitionBytes	128MB	128-256MB
spark.serializer	JavaSerializer	org.apache.spark.serializer.KryoSerializer

Final note None of these values are universal. They are starting points; the measurement loop in section 9 is what actually finds the right number for a given job and cluster.